Ruprecht-Karls-Universität Heidelberg
Institut für Informatik
Artificial Intelligence for Programming

Bachelorarbeit

PandaRec - Code Recommendation System for the Pandas Library based on Recipes

PandaRec - Code-Empfehlungssystem für die Pandas-Bibliothek auf Basis von Rezepten

Name:	Frederick Vandermoeten
Matrikelnummer:	4055264
Betreuer:	Prof. Dr. Artur Andrzejak
Datum der Abgabe:	19.10.2023

Ich versichere hiermit, dass ich diese Bachelor-Arbeit selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Sowohl inhaltlich als auch wörtlich entnommene Inhalte wurden als solche kenntlich gemacht. Die Arbeit ist in gleicher oder vergleichbarer Form noch bei keiner andere Prüfungsbehörde eingereicht worden.

Heidelberg, dd.mm.yyyy

Zusammenfassung

Dies ist eine Zusammenfassung der Arbeit.

Abstract

This is the abstract.

Contents

List of Figures 1

List of Tables 2

List of Listings 3

1 Introduction 5

1.1 Motivation . . . 5
1.2 Problems . . . 5
1.3 Solutions . . . 5
1.4 Structure of this Thesis . . . 6

2 Background 7

2.1 Pandas . . . 7
2.2 Natural Language Processing (NLP) . . . 7
2.3 Code Recommendation Systems . . . 7
2.4 Existing Code Recommendation Systems . . . 8
2.4.1 Code Completion . . . 8
2.4.2 Code Generation . . . 8
2.4.3 Generative AI . . . 9
2.5 Searching for Code . . . 9
2.5.1 Lemmatization and Stemming . . . 9
2.5.2 Lexical Search . . . 9
2.5.3 Semantic Search . . . 10
2.5.4 Bidirectional Encoder Representations from Transformers (BERT) 10
2.6 Normalized Discounted Cumulative Gain (NDCG) . . . 11
2.7 Existing Datasets . . . 11

3 Approach 12

3.1 Recipes . . . 12

3.2 The Recommendation System 14
3.2.1 Context 14
3.2.2 The Recommendation Engine 15
3.2.3 Ranking Strategies 15
3.3 The Jupyter Notebook GUI 24
3.3.1 The Data Grid and Search Box 25
3.3.2 The Search Results 25
3.3.3 Updating Results 25
3.3.4 Applying the Code Snippet 26
3.3.5 Observing the DataFrame 26

4 Evaluation **27**
4.1 Setup and Search Delays 27
4.2 Algorithm Accuracy 28
4.2.1 Creating the Ranking Dataset 28
4.2.2 Ranking Based on the Autogenerated Recipes 28
4.2.3 Comparing Semantic Search Models 29
4.3 Working With Jupyter Notebooks 30

5 Conclusion **31**
5.1 Future Work 31

Bibliography **32**

List of Figures

2.1 Bi vs Cross-Encoder 10

3.1 Recommender Widget 24

List of Tables

4.1 Search Delay 28
4.2 NDCG Scores 29
4.3 NDCG Scores 29

List of Listings

3.1 Recipe Dataclass. 12
3.2 Generate Basic Recipes. 13
3.3 Context Class. 14
3.4 Recommender Class. 15
3.5 RankingStrategy Class. 16
3.6 Name Search. 16
3.7 Fuzzy Search. 17
3.8 Index Search Constructor. 19
3.9 Lemmatization. 19
3.10 Building the Inverted Index. 20
3.11 Searching the Inverted Index. 21
3.12 Embedding the Recipes. 22
3.13 Searching the Embeddings. 23
3.14 OpenAI embeddings. 23
3.15 Observe Callbacks. 25
3.16 Copy Button. 26

4.1 Timing index search with a saved index. 27

1 Introduction

1.1 Motivation

(NOTE: title page has institut in german and lehrstuhl in english, ask prof what i should write)

Pandas is a library for Python that provides data analysis and manipulation tools [13]. While it is powerful and flexible, it is equally complex.

Researchers from all fields often have to work with data. However, researchers or users new to data analysis can struggle to learn how to use pandas or similar data manipulation software since searching documentation and tutorials is time-intensive. If the user has limited programming knowledge, this problem gets exacerbated since they must learn multiple things simultaneously.

Code recommendation systems are a staple in modern software development. Different methods can help developers write code faster and more efficiently, increase code clarity and maintainability, and help with error detection.

1.2 Problems

A recommendation system for the pandas library should be fast, easy to set up, easy to use, and useful to all users. That means it should provide the user with relevant recommendations even if they still need to gain domain knowledge. However, current code recommendation only fulfill some of the described requirements.

1.3 Solutions

My solution uses a Jupyter Notebook GUI that anyone can easily set up in the browser. I will use a set of "recipes" code snippets with at least a name and a description.

In the pursuit of developing a sound code recommendation system, we will compare

different methods of recommending code and implement them modularly so that methods can be easily exchanged and even composed out of multiple methods.

1.4 Structure of this Thesis

In the second chapter, I will present the current state of recommendation systems for pandas and the techniques that I will use in my recommendation system. The third chapter deals with the design of the GUI and modular recommendation system and then the implementation of the recipes and different algorithms. In the fourth chapter, I will evaluate how the specific ranking algorithms and the different recipe datasets perform compared to each other and other recommendation systems.

2 Background

This chapter will introduce the needed background knowledge to the reader. It will introduce the Pandas library, Code Recommendation Systems, how to search for code, and how to rank our recommendation algorithms.

2.1 Pandas

Pandas is an open-source library for the Python programming language that provides data analysis and manipulation tools. The core of the library are the Series and DataFrame data structures. Series are one-dimensional labeled arrays of any type. DataFrames are two-dimensional data structures that function like a table with rows and columns. Pandas provides tools for data cleaning, manipulation, analysis, and exploration of these data structures. Additionally, tools for data import and export are provided. Pandas also pairs well with visualization libraries like Matplotlib.

2.2 Natural Language Processing (NLP)

Natural language processing is a subfield of artificial intelligence that interprets and generates natural (human) language. In our specific case, we want to query in natural language since this is easier for a new user.

Let us look at existing recommendation systems so we can get an understanding of what our solution should be capable of.

2.3 Code Recommendation Systems

Code recommendation systems help developers write code more efficiently. They can suggest code snippets, highlight errors, and give quick access to documentation. As described by Robillard et al. 2010 [1], code recommendation systems have these design dimensions:

Context is the input of the recommendation system. This can include a large amount of different information. We can look at what the user is typing, the file we are in, the codebase in general, and the interaction history. For pandas, we can also look at the current DataFrame and its column- and row names, datatypes, and with a GUI, what part of the DataFrame the user has highlighted.

The Recommendation Engine uses the context and other data sources, such as data from different repositories and bug reports, to rank the recommendation output.

Output can be provided either on demand or continuously and can be multiple suggestions or a single (often inline) suggestion.

User feedback can change the system in many ways. The user can provide feedback by changing the context and adapting the recommendation engine locally or globally, where feedback either changes the output of the current user or all users.

2.4 Existing Code Recommendation Systems

The code recommendation system we want should be fast, easy to set up, use, and useful to all users. So, let us look at a few existing code recommendation systems and see how they are inadequate for all our goals.

2.4.1 Code Completion

Code completion is already a part of most IDEs. It is a form of autocomplete. In its most usual form, after the user starts typing, it will recommend multiple completions of the typed word. While it helps to reduce typos, it cannot help a new user use a library since basic knowledge of the different functions is required. Code completion makes an experienced user more efficient, but more is needed for inexperienced users. Additionally, it requires an IDE, which will be an additional piece of software that someone who is unfamiliar with computer science would have to learn and set up.

2.4.2 Code Generation

Code recommendation systems can generate entire blocks of code. This can be used for boilerplate code or whole functions. While automated boilerplate cannot help with our use case, there are code generation systems that meet quite a lot of our requirements.

2.4.3 Generative AI

(NOTE: Maybe get rid of this, or move before semantic, or redo) The Transformer [4] has been shown to perform well on natural language processing tasks [5]. Multimodal models like ChatGPT [14] and GitHub Copilot [**copilot**] are already commonly used to generate code. Using ChatGPT or GitHub Copilot by writing query-like comments performs decently on this task. Some problems are the cost of Copilot and the requirement to be online. How good they are will be discussed in the Chapter 4.

2.5 Searching for Code

Since my approach leverages "recipes", a corpus of common data manipulation tasks, I can use search algorithms to search through the descriptions of the recipes. Search algorithms that search for occurrences of the literal query are called lexical search algorithms.

2.5.1 Lemmatization and Stemming

Since words can have different inflections, e.g., walked and walking, they need to be brought to a generic form. Stemming solves this by removing common suffixes. This can, however, result in stems that are not actual words. Lemmatization ensures that the resulting generic form is a valid dictionary entry. It requires linguistic knowledge and a complete vocabulary. Lemmatization is therefore more computationally intensive than stemming but provides more meaningful results.

2.5.2 Lexical Search

In lexical search, the search engine looks for literal matches of the query words [3]. A common and effective way to implement lexical search is with search indexing.

Search Indexing builds an inverted index out of the corpus. First, a forward index is built, which stores word occurrences per document. Then, the inverted index can be constructed, which stores for each word all the annotations that contain it [2].

With this index, the search algorithm can quickly retrieve a list of annotations containing the query.

2.5.3 Semantic Search

Semantic search can be described as a search for meaning [3]. It aims to understand the meaning of a query instead of literal matches. The transformer [4] is a deep learning architecture that can capture relationships between words and phrases, which enables context-aware applications. For this reason, transformer-based models such as the Generative Pre-trained Transformer (GPT) and Bidirectional Encoder Representations from Transformers (BERT) perform well on natural language tasks.

2.5.4 Bidirectional Encoder Representations from Transformers (BERT)

BERT [8] and the more extensively trained RoBERTa [6] are natural language processing models based on the transformer architecture. They were trained by predicting missing words (masked language model) in sentences from both left and right contexts (Bidirectional). This bidirectionality lends itself to sentence pair regression, like sentence similarity. With BERT, both sentences are passed into the model simultaneously, and the similarity is predicted. This is called cross-encoding and produces similarity scores with great accuracy. However, when cross-encoders compute the similarity between a query and a corpus, they will have to do a cross-encoding for every element of the corpus.

To combat this, Sentence-BERT [7] uses a siamese network structure. Instead of computing the similarity, we compute the sentence embeddings, which can then be compared, often via cosine-similarity or the dot product [15].

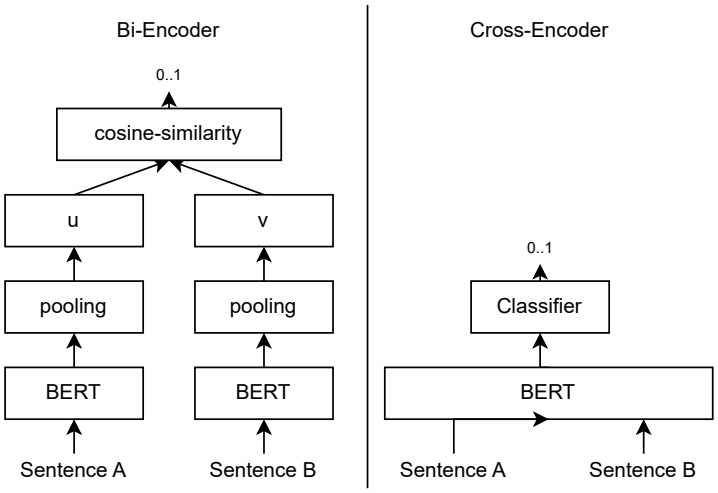


Figure 2.1: Bi vs Cross-Encoder. Taken from [15].

2.6 Normalized Discounted Cumulative Gain (NDCG)

I will evaluate the different algorithms using the Normalized Discounted Cumulative Gain (NDCG). It is a standard quality measure often used for ranking recommendation systems that score the first k results compared to an ideal result.

We first need the gains for each ranking item to calculate the cumulative gain. These can be either manually assigned or created with a more performant model. These scores can be either clamped to 0 and 1 or continuous in that interval.

To calculate the cumulative gain (CG), we take the sum of all the individual gains up to the last rank k.

$$CG = \sum_{i=1}^{k} gains$$

This does not take into account the order of the results, so we discount the gains by rank to get the discounted cumulative gain (DCG).

$$DCG = \sum_{i=1}^{k} \frac{gains}{log_2(i+1)}$$

Lastly, we divide the discounted cumulative gain by the ideal discounted cumulative gain (IDCG), where everything is ranked in an optimal way to get the normalized discounted cumulative gain.

$$NDCG = \frac{DCG}{IDCG}$$

Since the last rank k is crucial, we usually write NDCG@k, e.g., NDCG@10.

2.7 Existing Datasets

We can use existing datasets to fine-tune our models and to evaluate our search algorithms. There is no ready-made dataset for the pandas library, but potentially useful datasets include CodeSearchNet(NOTE:CITE), a dataset of functions and their docstrings, and the StackOverflow data.

3 Approach

This chapter will go over my solution. My solution's three major building blocks are the recipes, the recommendation system, and the GUI. It will also go over the different ranking strategies in detail.

3.1 Recipes

Recipes are a data structure consisting of a code snippet, a description, a name, and optionally keywords or argument type information.

The name and description field should give the user information about what the recipe is for. The code snippets themselves could be common functions for data manipulation, plotting, and data import and export.

The Recipes are implemented as a Python data class since they manage data, not logic.

```
1 # pandaRec/recipe.py
2
3 @dataclass
4 class Recipe:
5     name: str
6     description: str
7     code: str
8     keywords: str
```

Listing 3.1: Recipe Dataclass.

The recipes are bundled into a dataset and saved inside a .json file, together with the dataset description and links to other resources I will discuss later.

There are multiple ways to generate these recipes. One could:

- Generate recipes with the dir() method.

- Modify already existing snippets, e.g., from here: [12].
- Collect pandas usage data from publicly available sources (Stackoverflow, CodeSearchNet) and search for often used snippets.
- Create snippets from publicly available usage guides for pandas.

I used the first method to generate a basic set of recipes like this:

```
# generate_recipes.ipynb

# modified from https://stackoverflow.com/a/34452
def get_methods(object):
    method_list = []
    for method in dir(object):
        # make sure its a function
        if not callable(getattr(object, method)):
            continue
        method_name = str(method)

        # ignore internal functions
        if method_name.startswith('_'):
            continue

        # only take functions with a docstring
        if str(getattr(object, method).__doc__) == 'None':
            continue
        method_list.append(
            [
                method_name,
                str(getattr(object, method).__doc__)
            ]
        )
    return method_list

recipes = [
    Recipe(method[0], method[1].strip(), "df." + method[0], "")
    for method in get_methods(df)
]
```

Listing 3.2: Generate Basic Recipes.

I also used the fourth method on the pandas documentation, specifically "10 minutes to pandas" and the "cookbook" from the pandas user guide (NOTE: cite link).

For the recipes I created, I followed the VSCode Snippet syntax for placeholder to make a later integration into VSCode easier. More on this in Section 5.1.

3.2 The Recommendation System

3.2.1 Context

As described in Section 2.3, the context is the system's input. My system collects the user query, along with the specified DataFrame and selections, rectangular areas in the DataFrame that the user selected.

```python
# pandaRec/context.py
from dataclasses import dataclass


@dataclass
class Selection:
    r1: int
    c1: int
    r2: int
    c2: int


class Context:
    selections: list[Selection]
    data: DataFrame
    query: str

    def __init__(...): ... # basic constructor, left out for brevity

    def is_whole_row(self, selection: Selection) -> bool:
        return selection.c1 == 0 and selection.c2 == self.data.shape[1]

    def is_whole_column(self, selection: Selection) -> bool:
        return selection.r1 == 0 and selection.r2 == self.data.shape[0]
```

Listing 3.3: Context Class.

I also provide functions `is_whole_row()` and `is_whole_column()` so one can check whether a whole row/ column is selected.

3.2.2 The Recommendation Engine

My recommendation system uses a strategy pattern for the recommendation algorithm. This makes it easy to implement multiple algorithms.

The recommender class, therefore, only holds the context, the current recommend strategy, and the list of recipes. Additionally, the strategy result is saved in the recommender as an ordered list of RecipeResults, recipes with scores.

```python
# pandaRec/recommender.py
class Recommender:
    context: Context
    search_algorithm: RecommendStrategy
    recipes: list[Recipe]
    recommended_recipes: list[RecipeResult]

    def __init__(...): ... # basic constructor, left out for brevity

    def recommend(self): # use the strategies search function
        self.recommended_recipes = self.search_algorithm.search(
            self.context, self.recipes
        )

    def show_results(self) -> str:
        result = ""
        for recipe_result in self.recommended_recipes:
            result +=
                f"{recipe_result.recipe.name}\t\tScore: {recipe_result.score}\n"
        return result
```

Listing 3.4: Recommender Class.

I decided not to save a reference to the index of the corresponding recipe inside the RecipeResults because, this way, a strategy can return arbitrary recipes. Storing the result list as a list of recipe indexes and score tuples would eliminate the data duplication, but I prefer the flexibility my approach provides.

3.2.3 Ranking Strategies

For my solution, I implemented multiple ranking strategies to compare them. These can be swapped out easily because of the strategy pattern.

```python
# pandaRec/strategies.py
from abc import ABC, abstractmethod

class RankingStrategy(ABC):
    @abstractmethod
    def search(self, context: Context, recipes: list[Recipe]) -> list[RecipeResult]:
        pass
```

Listing 3.5: RankingStrategy Class.

Any strategy we add will inherit this abstract class and provide its search function. The recommender will call this function as shown in Listing 3.4. All my strategies score the recipes in the interval $[0; 1]$. Let us first look at a basic strategy to understand how this works.

Name Search

Name search searches for the query string in the name of the recipes, scoring it one if it finds it and not returning the recipe at all if the query string is absent. This ends up in name search working like code completion since the names of the auto-generated recipes are the same as the function names.

```python
# pandaRec/strategies.py
class NameSearch(RankingStrategy):
    @staticmethod
    def search(context: Context, recipes: list[Recipe]) -> list[RecipeResult]:
        result = []
        for recipe in recipes:
            if context.query in recipe.name:
            result.append(RecipeResult(1, recipe))
    return result
```

Listing 3.6: Name Search.

We can see with this example how a ranking algorithm will work; it will iterate over the recipes, score them, sort them (not the case here since the scores are all 1), and then return them to the recommender.

Fuzzy Search

Fuzzy search uses a string distance function to search for approximately similar strings. The Levenshtein distance is commonly used. In it, the distance amount increases by a constant for every insertion, deletion, and substitution.

$$[htb]lev(a,b) = \begin{cases} |a| & \text{if } |b| = 0 \\ |b| & \text{if } |a| = 0 \\ lev(tail(a), tail(b)) & \text{if } a[0] == b[0] \\ \min \begin{cases} lev(tail(a), b) & + \text{i} \\ lev(a, tail(b)) & + \text{d} \\ lev(tail(a), tail(b)) & + \text{c} \end{cases} & \text{else} \end{cases} \tag{3.1}$$

If i + d = c, this reduces to the InDel distance, which only counts insertions and deletions. Since performance starts to matter here, I have used the library "rapidfuzz" to calculate the distances. Their `partial_token_sort_ratio` function sorts the words and calculates the partial ratio between the prompt and the description.

(NOTE: funky code placement, fix after everything else is done)

```python
# pandaRec/strategies.py
class FuzzySearchDescription(RankingStrategy):
    def __init__(self, ratio=fuzz.partial_token_sort_ratio):
        super().__init__()
        self.ratio = ratio # let the user choose a different ratio if they want

    def search(self, context: Context, recipes: list[Recipe]) -> list[RecipeResult]:
        result = []
        for recipe in recipes:
            score = self.ratio(context.query, recipe.description,
                               processor=default_process)
            result.append(RecipeResult(score, recipe))
        result.sort(key=lambda recipe_result: recipe_result.score, reverse=True)
        return result
```

Listing 3.7: Fuzzy Search.

The partial ratio they use is implemented in C++ and runs for short queries (n<64), most queries will be this size, in worst-case O(nm) time, with m being the number of

different alignments of the shorter string in the bigger string, effectively the size of the bigger string. Since we must iterate over all recipes, we get O(nmr), which can incur noticeable slowdowns (in the decisecond range) on web-hosted Jupyter engines. So, let us look at an algorithm that still allows us to find partial matches and matches of similar words while simultaneously being way faster.

Index Search

I have already introduced index search in Section 2.5.2. We build the index when we construct the strategy, only if we do not provide a path to a prebuilt built index:

```python
# pandaRec/strategies.py
class IndexSearch(RankingStrategy):
    def __init__(self, recipes: list[Recipe], path: str = ""):
    super().__init__()
    if not path:
        self.index = generate_search_index(
            [recipe.description for recipe in recipes]
        )
    else:
        self.index = load_search_index(path)
```

Listing 3.8: Index Search Constructor.

This path can be saved alongside the list of recipes in the .json and is one of the paths I alluded to in Section 3.1.

Before generating the index, we first want to lemmatize the descriptions (see Section 2.5.1). For this, we start by tokenizing the descriptions with the regex `"\w*[a-zA-Z]\w*"`, which matches words with at least one letter. We then use the Natural Language Toolkit NLTK to lemmatize the words and remove common stop words like "a" or "the" that are irrelevant to the search.

```python
# pandaRec/search_index.py
def lemmatize_no_stop_words(text: str) -> list[str]:
    words = re.findall(r"\w*[a-zA-Z]\w*", text)
    wnl = WordNetLemmatizer()
    return [
        wnl.lemmatize(word)
        for word in words
        if word not in stopwords.words("english")
        # stopwords are common words like "a" that are not relevant for the search
        ]
```

Listing 3.9: Lemmatization.

We can then build the forward index and, lastly, the inverted index.

```python
# pandaRec/search_index.py
def generate_search_index(descriptions: list[str])
        -> dict[str, list[tuple[int, int]]]:
    # 1. Lemmatize the descriptions
    lemmatized_descriptions = [
            lemmatize_no_stop_words(description) for description in descriptions
        ]
    # 2. build the forward index, one list of (word, frequenzy) tuples
    # per description
    frequency_per_description = [
            [(word, description.count(word)) for word in set(description)]
            for description in lemmatized_descriptions
        ]
    # 3. construct the inverted index as hashmap with list of
    # (description_index, frequency) tuples for each word
    frequency_per_word = {}
    for idx, description in enumerate(frequency_per_description):
        for (word, count) in description:
            if word not in frequency_per_word:
                frequency_per_word[word] = [(idx, count)]
            else:
                frequency_per_word[word].append((idx, count))
    return frequency_per_word
```

Listing 3.10: Building the Inverted Index.

We can then pickle the hashmap to save it to disk. When searching for a query, we lemmatize it and normalize how many query words are in each document.

We also must sort the list of results since we want this list to be in the order of ranking.

```python
# pandaRec/strategies.py
class IndexSearch(RankingStrategy):
    ...

    def search(self, context: Context, recipes: list[Recipe]) -> list[RecipeResult]:
        lemmatized_search = lemmatize_no_stop_words(context.query)

        # add 1 to the score for each word that is in the description (per recipe)
        scores = len(recipes) * [0]
        for word in lemmatized_search:
            if word in self.index:
                for idx, _ in self.index[word]:
                    scores[idx] += 1

        num_words = len(lemmatized_search)
        result = [
                RecipeResult(score / num_words, recipe) # normalize score
                # scores and recipes are in the same order so we can zip() them
                for score, recipe in zip(scores, recipes)
            ]
        result.sort(key=lambda recipe_result: recipe_result.score, reverse=True)
        return result
```

Listing 3.11: Searching the Inverted Index.

Semantic Search

The basics of semantic search were already described in Section 2.5.3. To implement a semantic search, we will use the Sentence-Transformers library, which is originally based on [7]. Like constructing the reverse index in the index-based search, we first must create embeddings for all our recipes. This is again handled in the constructor.

```
        # pandaRec/strategies.py
class SemanticSearch(RankingStrategy):
    def __init__(self, recipes: list[Recipe], path: str = "",
                 model: str = "sentence-transformers/all-mpnet-base-v2"):
    super().__init__()
    self.model = SentenceTransformer(model)
    descriptions = [recipe.description for recipe in recipes]
    if not path:
        self.embeddings = self.model.encode(descriptions)
    else:
        self.embeddings = load_embeddings(path)
```

Listing 3.12: Embedding the Recipes.

We can then embed our query and use the cosine similarity to search for descriptions with similar semantic meanings. I have tested this algorithm with three different models: "all-mpnet-base-v2" and "all-MiniLM-L6-v2" which are general general-purpose, and "krlvi/sentence-t5-base-nlpl-code_search_net" which is trained with the "code_search_net" dataset (NOTE:cite). In Section 4.2.3 I will evaluate the differences between these models.

OpenAI Embeddings

We can also use OpenAIs API to generate embeddings. Their util function `get_embeddings` automatically manages batching, but the process is other than that the same as with Sentence-Transformers. The ranking process is the same as in Listing 3.13.

```python
# pandaRec/strategies.py
class SemanticSearch(RankingStrategy):
    ...

    def search(self, context: Context, recipes: list[Recipe]) -> list[RecipeResult]:
        query_embedding = self.model.encode(context.query, convert_to_tensor=True)
        cos_scores = [
            util.cos_sim(query_embedding, embedding).item(
            for embedding in self.embeddings
        ]
        result = [
            RecipeResult(score, recipe)
            for recipe, score in zip(recipes, cos_scores)
        ]
        result.sort(key=lambda recipe_result: recipe_result.score, reverse=True)
        return result

```

Listing 3.13: Searching the Embeddings.

```python
# pandaRec/strategies.py
class OpenAIEmbeddings(RankingStrategy):
    def __init__(
        self,
        recipes: list[Recipe],
        path: str = "",
        model: str = "text-embedding-ada-002",
    ):
        super().__init__()
        self.model = model
        descriptions = [recipe.description for recipe in recipes]
        if not path:
            self.embeddings = get_embeddings(descriptions, engine=self.model)
        else:
            self.embeddings = load_embeddings(path)
```

Listing 3.14: OpenAI embeddings.

3.3 The Jupyter Notebook GUI

For my solution, I decided to go with a Jupyter Notebook GUI. Jupyter Notebooks are already used extensively in machine learning and data science since they have many essential strengths in these fields. Being able to run code cell-by-cell, one can quickly explore and visualize data. The capability to render markdown also helps with documentation. Lastly, services like google colab allow for running notebooks directly in the browser without extensive setup.

(NOTE: Maybe change to a screenshot that shows off the code field better)

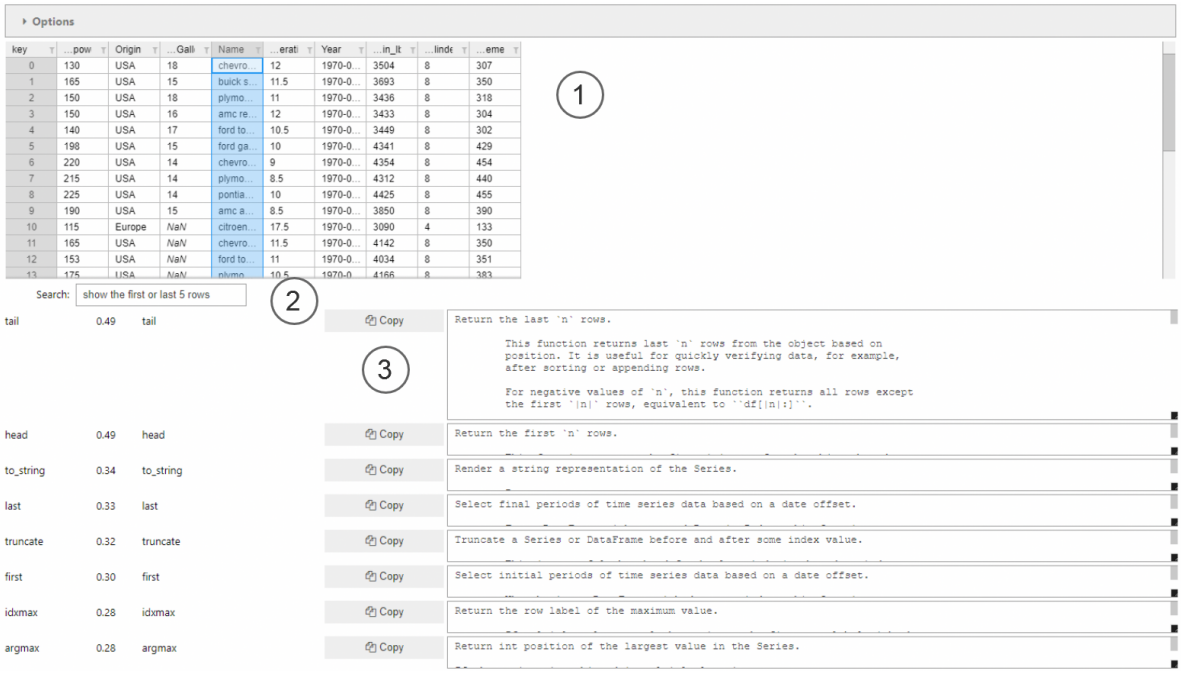


Figure 3.1: Main parts of the recommender.

The GUI is made of three main building blocks:

1. The data grid.
2. The search box.
3. The search results.

There also exists an options dropdown menu.

3.3.1 The Data Grid and Search Box

The data grid uses the library ipydatagrid(NOTE:cite), a data grid widget that is recommended by the Jupyter Tutorial(NOTE:cite). It allows for scrolling, resizing, and selecting, which is more than enough for our use case.

The search box is a Text widget.

3.3.2 The Search Results

The search results are shown in a vertical list. Each result has, in order, a name, score, code snippet, copy button, and description. The descriptions are resizable to allow for longer descriptions. The user can change the length of the list in the options.

3.3.3 Updating Results

To update the results, I used the observe methods of the various widgets to register a callback.

```
1         # pandaRec/panda_rec_widget.py
2         class PandaRecWidget(widgets.VBox):
3         def __init__(...):
4         ...
5         self.search_box.observe(self.update_recommendations, names="value")
6         self.data_grid.observe(self.update_recommendations, names="selected_cells")
7
8         def update_recommendations(self, _change):
9         self.recommender.set_search(str(self.search_box.value))
10        self.recommender.set_selection(self.data_grid.selected_cells)
11        self.recommender.recommend()
12        self.result_widget.update(self.recommender.recommended_recipes)
```

Listing 3.15: Observe Callbacks.

This callback then updates the context of the recommender, recommends recipes, and changes the search results. There is a consideration here about whether we want the callback to be triggered anytime a change is made or only after a short duration of inactivity. This could result in additional costs, especially for the OpenAI Embeddings since we send more requests than necessary. I decided not to implement this since, as we will see, all algorithms are quick. If needed, however, one could follow the Jupyter

Widgets Documentation to create a debouncing decorator(NOTE:cite). (NOTE: I did this by now, update)

3.3.4 Applying the Code Snippet

We could apply code snippets to the underlying DataFrame in two main ways. We could use `eval()` to directly evaluate the code snippet or copy it to a new cell directly or via user input. The `eval()` approach makes changing, e.g., row names before evaluating, more complicated and will also lose Jupyter notebooks upside of cell-by-cell execution. By copying the snippet to the clipboard, we allow the user to choose what to do with it.

When using the notebook online, we cannot easily access the clipboard with the Pyperclip library. That is why I created a copy button with `ipywidgets.HTML`, which allows for custom HTML input.

```
# pandaRec/copybutton.py
import ipywidgets as widgets

def copy_button_html(copytext=""):
    base_html = """
    <button class="copybutton" onclick=navigator.clipboard.writeText('{}')>
        Copy
    </button>
    """
    return base_html.format(copytext)

```

Listing 3.16: Copy Button.

4 Evaluation

This chapter evaluates the different ranking algorithms concerning their setup speed, search speed, and ranking performance. It will also compare the different recipe datasets.

4.1 Setup and Search Delays

To evaluate delays in the system, we can use Jupyter notebooks inbuild %timeit magic command, which uses the timeit module to time the execution of a statement. As an example, this is what timing the setup time for index search with a saved index looks like:

```
%timeit IndexSearch(recipes, '../recipes/search_index.pkl')

# 1.50 ms ± 3.07 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

Listing 4.1: Timing index search with a saved index.

If we do this for all implemented algorithms, we can compare their setup and search time. I have used the query "fill missing values with zero" to analyze search time.

Name search and fuzzy name search both do not have any setup, and fuzzy description search only registers the fuzzy ratio algorithm, which results in basically instant setup time. Index search takes about 8 seconds to build the index. The embedding using Sentence Transformers takes about one and a half minutes to embed all the descriptions. This could be sped up by using the GPU by passing `device="cuda"` to the SentenceTransformer constructor. The OpenAI Embedding is batched and calculated remotely, which should keep the setup time decoupled from the hardware.

Name	Setup Time	Search Time
Name Search	68.0 ns ± 0.2 ns	6.53 µs ± 0.08 µs
Fuzzy Name Search	68.4 ns ± 0.5 ns	756.0 µs ± 1.3 µs
Fuzzy Description Search	276.0 ns ± 0.8 ns	10.00 ms ± 0.05 ms
Index Search	7.87 s ± 0.08 s	1.74 ms ± 0.01 ms
Semantic Search	85 s ± 1 s	38.2 ms ± 0.9 ms
OpenAI Embedding	3 s ± 1 s	668 ms ± 949 ms
Saved Index/ Embedding		
Index Search	1.50 ms ± 0.03 ms	-
Semantic Search	682 ms ± 10 ms	-
OpenAI Embedding	27.50 ms ± 0.25 ms	-

Table 4.1: Setup and search delay for all implemented algorithms.

4.2 Algorithm Accuracy

To evaluate the accuracy of the algorithms, we will use the NDCG introduced in Section 2.6.

4.2.1 Creating the Ranking Dataset

There are no good pre-existing datasets for ranking the algorithms, so I will create my own. To do this, I selected the article (NOTE: cite article). For every one of the thirty functions, I created a description and selected recipes that can fulfill these goals (NOTE: weird wording), which I assigned the score one. I then calculated the NDCG@5 for each of the algorithms. For some of the descriptions, none of the recipes can fulfill the goal. These are not counted towards the NDCG@5 score (since the IDCG@5 is 0, breaking our formula). Here is an overview of how many of the descriptions cannot be achieved by each recipe dataset:

(NOTE: Table of nonachievements)

4.2.2 Ranking Based on the Autogenerated Recipes

Since none of the ranking descriptions are substrings of the recipes' names, Name Search does not recommend anything and, therefore, always scores zero. This is fine since it worked more as a proof of concept, but I will omit it in the following tables.

I was expecting both the lexical searches and the embeddings to work well on the autogenerated recipes since they have a decently long description, with a mean length

of 2221 characters and a mean amount of 71.6 lines.

Name	Score
Name Search	0
Fuzzy Name Search	0.39
Fuzzy Description Search	0.26
Index Search	0.21
Semantic Search	0.62
OpenAI Embedding	0.54

Table 4.2: NDCG Scores for all implemented algorithms.

The Lexical search algorithms work noticably (NOTE:finish)

4.2.3 Comparing Semantic Search Models

Now, let us compare the different models I considered for the semantic search using Sentence-Transformers.

all-mpnet-base-v2 is the general-purpose model from Sentence-Transformers with the best quality.

all-MiniLM-L6-v2 is a smaller general-purpose model that the Sentence-Transformer documentation claims is five times faster at the cost of some performance.

sentence-t5-base is the base model of the t5 architecture, which is trained on predicting multiple masked words simultaneously.

krlvi/sentence-t5-base-nlpl-code_search_net is based on a t5-base checkpoint and trained on the "code_search_net" dataset.

Name	Score	Setup Delay	Search Delay
all-mpnet-base-v2	0.62	$85\,\text{s} \pm 1\,\text{s}$	$38.2\,\text{ms} \pm 0.9\,\text{ms}$
all-MiniLM-L6-v2	0.50	$10.3\,\text{s} \pm 0.1\,\text{s}$	$16.4\,\text{ms} \pm 0.6\,\text{ms}$
sentence-t5-base	0.53	$52.3\,\text{s} \pm 0.5\,\text{s}$	$40.6\,\text{ms} \pm 9.2\,\text{ms}$
sentence-t5-base-...	0.48	$50.8\,\text{s} \pm 0.5\,\text{s}$	$39.6\,\text{ms} \pm 1.1\,\text{ms}$

Table 4.3: NDCG Scores for different models.

Using all-MiniLM-L6-v2 resulted in a 8.3× speedup in setup delay and a 2.3× speedup in searching delay. The model trained on "code_search_net" is noticeably worse than

even. This is confirmation that the "code_search_net" will not help much in training a good model.

4.2.4 Performance of the Different Recipe Datasets

ranking different recipe datasets (NOTE: do this)

4.3 Working With Jupyter Notebooks

I decided to implement my solution in Jupyter Notebooks. Notebooks in combination with

4.3.1 Observing the DataFrame

When we change the DataFrame with either a copied function or, in general, we would like to automatically update the recommender's DataFrame and the data grid. However, this is currently not supported by the widgets library (CITE:https://github.com/bloomberg/ipydatagr

reflection: no observe on dataframes

outlook: fine-tune, use

(NOTE: copy results from notebook to here)

bad: no observe on dataframes

5 Conclusion

5.1 Future Work

semantic search + documentation = good combo future work: fine tuning and adding NL based on row/ line names [11] [10] [9] porting to vscode implementing a more advanced web based algorithm

Bibliography

[1] Martin Robillard, Robert Walker, and Thomas Zimmermann. "Recommendation Systems for Software Engineering". In: *IEEE Software* 27.4 (2010), pp. 80–86. DOI: 10.1109/MS.2009.161.

[2] Abhishek Das and Ankit Jain. "Indexing the world wide web: The journey so far". In: *Next Generation Search Engines: Advanced Models for Information Retrieval*. IGI Global, 2012, pp. 1–28.

[3] Hannah Bast, Björn Buchhold, Elmar Haussmann, et al. "Semantic search on text and knowledge bases". In: *Foundations and Trends® in Information Retrieval* 10.2-3 (2016), pp. 119–271.

[4] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[5] Alec Radford et al. "Improving language understanding with unsupervised learning". In: (2018).

[6] Yinhan Liu et al. "Roberta: A robustly optimized bert pretraining approach". In: *arXiv preprint arXiv:1907.11692* (2019).

[7] Nils Reimers and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks". In: *arXiv preprint arXiv:1908.10084* (2019).

[8] Zhangyin Feng et al. "Codebert: A pre-trained model for programming and natural languages". In: *arXiv preprint arXiv:2002.08155* (2020).

[9] Tianyu Gao, Xingcheng Yao, and Danqi Chen. "Simcse: Simple contrastive learning of sentence embeddings". In: *arXiv preprint arXiv:2104.08821* (2021).

[10] Kexin Wang, Nils Reimers, and Iryna Gurevych. "Tsdae: Using transformer-based sequential denoising auto-encoder for unsupervised sentence embedding learning". In: *arXiv preprint arXiv:2104.06979* (2021).

[11] Kexin Wang et al. "Gpl: Generative pseudo labeling for unsupervised domain adaptation of dense retrieval". In: *arXiv preprint arXiv:2112.07577* (2021).

[12] Raza Balbale. *Basic Pandas Snippets*. Sept. 2023. URL: https://github.com/snippington/snp-pandas-basic/blob/main/snippets/pandas-basic.code-snippets.

[13] Pandas Community. *Pandas Documentation*. Sept. 2023. URL: https://pandas.pydata.org/docs/.

[14] OpenAI. *ChatGPT Website*. Sept. 2023. URL: https://chat.openai.com/.

[15] Nils Reimers. *Sentence-Transformers Documentation*. Sept. 2023. URL: https://www.sbert.net/examples/applications/cross-encoder/README.html.